

Skill 1

Linux VM, GCC, Git Repository, Project Structure, Shell Architecture and Makefile

Aim

To install and configure a Linux virtual machine, set up the GCC compiler and Git, create a structured project repository, understand the basic architecture of the Linux shell, and build an initial Makefile for compiling the project.

Requirements

- Ubuntu Linux
- VirtualBox or VMware
- GCC Compiler
- Git
- Terminal
- Text editor / VS Code

Step-by-Step Implementation

Step 1: Install Ubuntu Virtual Machine

1. Install VirtualBox or VMware on the host system.
2. Create a new virtual machine.
3. Allocate suitable RAM and storage.
4. Attach the Ubuntu ISO image.
5. Start the virtual machine and complete the Ubuntu installation.
6. Open the Ubuntu Terminal after installation.

Step 2: Update the System

```
sudo apt update  
sudo apt upgrade
```

Step 3: Install GCC and Development Tools

```
sudo apt install build-essential
```

Verify GCC:

```
gcc --version
```

Step 4: Install Git

```
sudo apt install git
```

Verify Git:

```
git --version
```

Step 5: Configure Git

```
git config --global user.name "Your Name"  
git config --global user.email "your@email.com"
```

Step 6: Create the Project

```
mkdir LinuxProcessManagement  
cd LinuxProcessManagement
```

Step 7: Create Project Structure

```
mkdir src docs data results reports  
touch README.md Makefile
```

The structure becomes:

```
LinuxProcessManagement/  
├── src/  
├── docs/  
├── data/  
├── results/  
├── reports/  
├── README.md  
└── Makefile
```

Step 8: Initialize Git Repository

```
git init
git status
```

Step 9: Understand Shell Architecture

The basic shell execution flow is:

```
User
  ↓
Terminal
  ↓
Shell
  ↓
System Calls
  ↓
Linux Kernel
  ↓
Hardware / Resources
```

The shell accepts commands from the user, interprets them and requests the operating system kernel to perform the required operations.

Step 10: Create Initial Makefile

Example:

```
CC = gcc
CFLAGS = -Wall -Wextra

TARGET = program
SRC = src/main.c

$(TARGET): $(SRC)
    $(CC) $(CFLAGS) $(SRC) -o $(TARGET)

clean:
    rm -f $(TARGET)
```

The Makefile provides an automated method for compiling and cleaning the project.

Observation

The Ubuntu environment was successfully configured with GCC and Git. A structured project repository was created and initialized using Git. The shell architecture and interaction between the user, shell, system calls and kernel were studied. An initial Makefile was created to automate compilation.

End Result

The Linux development environment was successfully prepared. GCC, Git, the project directory structure and the initial Makefile were configured successfully.